

Aula 4: Servidor, Docker, Kamal e deploy

Você sai daqui com: a sua API publicada em `https://seunome.test`, empacotada em Docker, servida com HTTPS, e com deploy e rollback num comando. **Gabarito:** `cd ~/capacitacao-gabarito && git checkout aula-04`

Este roteiro é a versão para a turma do guia de infraestrutura do `seem-backend`. As decisões são as mesmas; a máquina é menor, e hoje ela mora no seu notebook.

O último dia

Você tem uma API com cinco fluxos de autenticação, testada, rodando com `bin/rails server`. **Hoje ela deixa de ser "um comando que eu rodo" e vira um sistema publicado:** sobe sozinha, aceita HTTPS, guarda os dados num lugar que sobrevive a reinício, e você consegue atualizar e voltar atrás sem ninguém perceber.

As ferramentas que fazem isso têm nome, e você vai conhecer cada uma no caminho: Docker empacota, um registry distribui, o Kamal publica, e um proxy termina o HTTPS.

E o servidor de hoje é **a sua própria máquina**. Não porque seja um faz de conta: é porque tudo o que você vai fazer aqui é idêntico ao que se faz numa máquina alugada, e num encontro de três horas o tempo é melhor gasto entendendo o deploy do que esperando um provedor liberar cota.

Duas coisas que ajudam:

Um passo mal feito só aparece três passos depois. Por isso toda prática tem uma linha **O que você deve ver**. Não siga com ela vermelha, mesmo que pareça que dá.

Hoje você vai encontrar mais erro do que nos outros encontros. É assunto novo e ferramenta nova, e a maioria dos erros não tem nada a ver com programar. É assim para todo mundo, sempre.

No fim do dia, na Prática 6, três comandos `curl` mostram na tela a diferença entre criptografia e confiança. São conceitos que costumam ficar embolados, e ali eles se separam.

1. O que é colocar no ar

O problema

`bin/rails server` funciona enquanto o seu terminal está aberto. Não é isso que um sistema em produção faz. Um sistema publicado precisa de:

- **subir sozinho** e continuar de pé sem ninguém olhando;
- **as mesmas versões** de tudo, sempre, em qualquer máquina;
- **atualizar sem sair do ar** quando você faz uma correção;
- **voltar atrás** quando a correção estava errada;
- guardar **segredo** de um jeito que não vaze no histórico do shell.

Tudo isso é o assunto de hoje, e nenhuma dessas cinco coisas depende de onde a máquina está.

As quatro opções de hospedagem

Opção	O que você controla	O que você opera
Hospedagem compartilhada	quase nada	nada
PaaS (Heroku, Render, Fly)	o app	nada, mas paga mais e obedece as regras deles
VPS	tudo: SO, firewall, banco	tudo
Kubernetes	tudo, em escala	muito mais

VPS = *Virtual Private Server*: um computador virtual dentro do servidor físico de um provedor, que é seu. Você tem acesso root, escolhe o sistema e instala o que quiser. É o que o `seem-backend` usa.

Por que uma VM com containers, e não Kubernetes

O `seem-backend` atende ~500 usuários num evento de uma semana. Kubernetes adicionaria custo e operação sem resolver um problema que existe. A escolha foi:

- menos peças para monitorar;
- deploy reproduzível com Docker e Kamal;
- banco e aplicação perto um do outro;
- recuperação simples numa máquina substituta;
- **evoluir só quando uma métrica real pedir.**

Escolher a ferramenta grande antes do problema grande é a forma mais cara de errar.

O que isso custa, e é honesto admitir: uma máquina só é ponto único de falha. Volume Docker não é backup. O Postgres não é gerenciado, e quem cuida dele é você.

Hoje: a sua máquina é o servidor

O Kamal não sabe onde a máquina está. Ele conecta por **SSH**, roda `docker` do outro lado, e pronto. Se o endereço for `127.0.0.1`, ele conecta na sua máquina; se for um IP público, na de lá. **É a mesma conexão e o mesmo arquivo de configuração.**

seu terminal —ssh→ a máquina que serve —docker→ os containers

Então é isto que muda quando você trocar por um servidor alugado:

	Hoje	Num servidor alugado
<code>SERVER_IP</code> no <code>.env</code>	<code>127.0.0.1</code>	o IP público
<code>KAMAL_SSH_USER</code>	o seu login	o usuário que o provedor criou
Certificado	autoassinado, e você confia nele na mão	emitido por uma CA pública
Antes disso	nada	provisionar a máquina: usuário, firewall, Docker
Quem tenta invadir	ninguém	bots, o dia todo, desde o primeiro minuto

O resto é idêntico: o `Dockerfile`, o `deploy.yml`, o registry, os segredos, o volume do banco, o zero-downtime, o rollback. É por isso que aprender aqui vale.

A seção **10** mostra o percurso numa máquina de verdade, e o apêndice `apendice-azure.md` tem o passo a passo completo para você fazer em casa.

2. A sua máquina vira o servidor

O Kamal precisa de duas coisas do outro lado: **um SSH que aceite a sua chave** e **o Docker rodando**. O Docker você já tem desde a Aula 1. Falta o SSH.

Onde você trabalha hoje: no mesmo lugar de sempre. Se você usa Windows, é dentro do **WSL2**, que é o seu Linux e vai ser o seu servidor. Se usa Linux ou macOS, é o próprio sistema. Você não vai instalar máquina virtual nenhuma.

Antes: duas chaves, um par

Criptografia assimétrica usa duas chaves que se completam. A **pública** você espalha; a **privada** nunca sai da sua máquina. O que uma fecha, só a outra abre.

Daí saem duas coisas diferentes:

- **sigilo:** eu fecho com a *sua* pública, e só você abre;
- **assinatura:** eu fecho com a *minha* privada, e todo mundo confere que fui eu.

É assim que o SSH funciona. Você põe a sua chave pública no servidor (`~/.ssh/authorized_keys`). Ao conectar, o servidor manda um desafio; você responde assinando com a privada; o servidor confere com a pública que já tinha. **A senha nunca trafega, nem existe.**

E é por isso que perder a chave privada é perder o acesso à máquina.

Compare com o JWT da Aula 3: lá a assinatura usa uma chave **simétrica** (HS256), porque quem assina e quem confere são o mesmo servidor.

Instalar o servidor de SSH

Você já usa o **cliente** de SSH (é o comando `ssh`). O que falta é o **servidor**, o programa que fica escutando na porta 22 e atende quem chega.

```
# Ubuntu, Debian e WSL2
sudo apt update && sudo apt install -y openssh-server
sudo service ssh start

# Fedora
sudo dnf install -y openssh-server && sudo systemctl enable --now sshd
```

No **macOS** não se instala nada: o servidor já vem no sistema, e você só liga em **Ajustes do Sistema** → **Geral** → **Compartilhamento** → **Sessão remota**.

Confira que ele está escutando:

```
ss -tlnp | grep :22      # Linux e WSL2
sudo lsof -i :22         # macOS
```

WSL2: o `sshd` não sobe sozinho a cada boot, a não ser que você habilite o systemd. Se depois de reiniciar o Windows o `ssh` parar de conectar, rode `sudo service ssh start` de novo. É o tropeço mais comum do dia, e a saída é uma linha.

O par de chaves da capacitação

Não reaproveite a chave do GitHub. Uma chave por finalidade: dá para revogar uma sem derrubar a outra.

```
ssh-keygen -t ed25519 -f ~/.ssh/capacita -C "capacita-servidor" -N ""
chmod 600 ~/.ssh/capacita
```

`chmod 600` significa "só o dono lê e escreve". O SSH **recusa** usar uma chave com permissão mais frouxa, e essa recusa é a primeira coisa a conferir quando aparecer `Permission denied (publickey)`.

Chave privada não vai por e-mail, não vai por WhatsApp, não vai para o Git. Nunca.

Autorizar a si mesmo

Agora a parte que costuma causar um sorriso: você põe a sua chave pública no `authorized_keys` da sua própria máquina. É exatamente o que você faria num servidor alugado, e é literalmente o mesmo comando.

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
cat ~/.ssh/capacita.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

E entre:

```
ssh -i ~/.ssh/capacita -o IdentitiesOnly=yes "$USER"@127.0.0.1
```

Você abriu uma sessão SSH da sua máquina para ela mesma. **Não é truque:** é uma conexão de rede de verdade, autenticada por chave, exatamente como a que o Kamal vai abrir daqui a pouco. Saia com `exit`.

O `IdentitiesOnly=yes` importa: sem ele o SSH oferece todas as chaves do seu agente, o servidor recusa depois de algumas, e você leva `Too many authentication failures` com a chave certa na mão.

Permissões, já que estamos aqui

Todo arquivo no Linux tem dono, grupo e três permissões: ler, escrever e executar.

```
chmod 600 arquivo    # o dono lê e escreve; mais ninguém vê nada
chmod 700 pasta      # o dono entra; mais ninguém
```

O SSH é rígido com isso de propósito: uma chave privada que o resto do sistema consegue ler não é uma chave privada. É a mesma lógica do `password_digest` da Aula 2, aplicada a arquivo.

E nunca rode a aplicação como `root`: o Dockerfile que você vai ver na próxima seção cria um usuário `rails` justamente para isso.

3. Docker

Imagem × container

Imagem é a receita: o sistema, o Ruby, as gems, o seu código, congelados. **Container** é o bolo: uma instância rodando. Uma imagem, muitos containers.

O Dockerfile do Rails

O Rails 8 gera um Dockerfile de produção pronto. Vale ler:

```
FROM ruby:3.4.9-slim AS base
# ...

FROM base AS build
RUN apt-get install -y build-essential libpq-dev ...
COPY Gemfile Gemfile.lock ./
RUN bundle install
COPY . .

FROM base
RUN groupadd --system --gid 1000 rails && useradd rails --uid 1000 --gid 1000
USER 1000:1000
COPY --from=build "${BUNDLE_PATH}" "${BUNDLE_PATH}"
COPY --from=build /rails /rails
```

Duas decisões importantes:

Multi-stage. O estágio `build` instala compilador e headers para compilar as gems nativas. A imagem final copia só o resultado. Compilador não vai para produção: menos peso e menos ferramenta à mão de quem invadir.

Usuário não-root. Se alguém escapar da aplicação, cai num usuário sem privilégio. É uma linha que muda o tamanho do estrago.

`.dockerignore`

Diz o que **não** entra na imagem: `.git`, `log/`, `tmp/`, `node_modules`. Sem ele a imagem fica gorda e, pior, o histórico do Git vai junto.

Registry

O registry é o "GitHub das imagens". Usamos o **ghcr.io** (GitHub Container Registry): já vem com a sua conta do GitHub.

O fluxo desta aula:

```
você builda → empurra a imagem para o ghcr.io → o servidor puxa e roda
```

Hoje quem builda e quem roda são a mesma máquina, e mesmo assim a imagem sobe para o GitHub e desce de lá. Parece rodeio, e é de propósito: é exatamente o que aconteceria com um servidor do outro lado do mundo, e é o que faz o mesmo `deploy.yml` servir nos dois casos sem mudar uma linha.

O token do registry

Fora do GitHub Actions não existe `GITHUB_TOKEN`. Crie um **Personal Access Token (classic)**:

GitHub → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token**, com os escopos `write:packages` e `read:packages`.

Guarde: é o seu `KAMAL_REGISTRY_PASSWORD`. Ele é mostrado **uma vez**.

Deixe o pacote público. Na conta grátis do GitHub, pacote **privado** tem uma cota de armazenamento pequena, e uma imagem Rails come quase toda ela. Pacote **público** é ilimitado e grátis, e não expõe o seu código: o repositório continua privado, e o que fica público é só a imagem já compilada.

Depois do primeiro deploy, em **github.com/SEU-USUARIO?tab=packages** → **automatic-auth-api** → **Package settings** → **Change visibility** → **Public**.

Um token com escopo de pacote e nada mais. É o mesmo princípio da chave SSH separada: quando vazar e um dia vaza, o estrago tem tamanho.

Arquitetura

O Kamal builda a imagem e depois a roda. Hoje as duas coisas acontecem na mesma máquina, então a arquitetura é a sua:

Seu notebook	SERVER_ARCH
Intel ou AMD	amd64
Mac com chip M1/M2/M3/M4	arm64

Descubra com:

```
uname -m      # x86_64 é amd64; aarch64 ou arm64 é arm64
```

4. Kamal

O Kamal (feito pela mesma turma do Rails) faz *zero-downtime deploy* com Docker, via SSH. Sem agente e sem painel: ele conecta na máquina que serve e roda `docker`.

O que ele faz num `kamal deploy`:

1. builda a imagem e empurra para o registry
2. conecta no servidor por SSH
3. puxa a imagem
4. sobe o container novo **ao lado** do antigo
5. espera o **health check** do novo passar, que é uma rota simples (a nossa é `/up`) que responde 200 quando a aplicação está de pé
6. manda o tráfego para o novo e derruba o antigo

Falhou o health check? Ele não troca. O antigo continua servindo.

`config/deploy.yml`, por partes

```
service: automic-auth-api
image: <%= ENV.fetch("GHCR_USER") %>/automic-auth-api
```

O arquivo é ERB: dá para usar `ENV`. É por isso que este `deploy.yml` é **igual para todo mundo da turma**, e é o mesmo que serviria numa VPS. O que muda são as variáveis.

```
ssh:
  user: ubuntu
  run_directory: /home/ubuntu/.kamal
```

Sem login root, então o Kamal precisa de um lugar para gravar lock e auditoria.

```
servers:
  web:
    hosts:
      - <%= ENV.fetch("SERVER_IP") %>
```

`SERVER_IP` é `127.0.0.1` hoje: a sua máquina. Num servidor alugado, é o IP público dele. Uma variável.

O bloco `ssh` também lê `SSH_PORT`, com padrão 22: é o que faz o Kamal funcionar sem mudança nenhuma para quem alcança a VM por encaminhamento de porta.

```
env:
  clear:
    SOLID_QUEUE_IN_PUMA: "true"
    WEB_CONCURRENCY: "1"
    DB_HOST: automic-auth-api-db
  secret:
    - RAILS_MASTER_KEY
    - JWT_SECRET
```

- `clear` vai como texto no comando `docker run`. Configuração, não segredo.
- `secret` vai por um arquivo de ambiente, com permissão restrita no servidor.

Colocar `JWT_SECRET` no `clear` seria o mesmo que publicá-lo: ele apareceria em `docker inspect` e no histórico do shell.

`DB_HOST: automic-auth-api-db` é o **nome do container** do banco. Containers na mesma rede Docker se enxergam por nome: não precisa de IP.

```
accessories:
  db:
    image: postgres:16
    directories:
      - data:/var/lib/postgresql/data
```

Accessory é um container que o Kamal gerencia mas não faz deploy junto com o app. O `directories` cria o volume: é ele que faz os dados sobreviverem a deploy e a rebuild.

`kamal deploy` **não** sobe accessories. Depois de mudar env de accessory: `kamal accessory reboot db`.

As três camadas de segredo

```
o seu shell (source .env)
  ↓ variáveis de ambiente na sua sessão
.kamal/secrets
  ↓ referencia as variáveis – nenhum valor aqui, por isso vai para o Git
config/deploy.yml (env.secret)
  ↓ o Kamal injeta no container
ENV["JWT_SECRET"] no Rails
```

Nenhum valor real toca o repositório em nenhum ponto. Num pipeline de CI a primeira camada vira os Secrets do GitHub Actions; **as outras três não mudam nada**.

5. Segredos do Rails

`credentials` e `master.key`

```
bin/rails credentials:edit
```

O Rails abre um YAML no editor, e ao salvar grava `config/credentials.yml.enc`: criptografado, seguro no Git. A chave que decripta é `config/master.key`, que **nunca** vai para o Git (já está no `.gitignore`).

Em produção, `master.key` vira a variável `RAILS_MASTER_KEY`.

Antes: o que é uma variável de ambiente

Um par `nome=valor` que o sistema operacional entrega ao processo quando ele sobe.

```
export JWT_SECRET=abc123
```

E o programa lê com `ENV["JWT_SECRET"]`.

Por que não deixar o valor no código? Porque o código vai para o Git. A ideia é: mesmo código, ambientes diferentes, valores diferentes. É um dos [12 fatores](#), e é como o Kamal injeta segredo no container.

Credentials × ENV

	Quando
Credentials	segredo estável, que é do app: chave de API de terceiro
ENV	o que muda por ambiente ou por máquina: senha do banco, host de SMTP

Este projeto usa ENV para quase tudo, porque quem provisiona (Kamal) já sabe injetar.

Gere o `JWT_SECRET`:

```
bin/rails secret
```

O seu `.env`

O Kamal lê o `.kamal/secrets`, que só **referencia** variáveis do seu shell. Quem põe valor nelas é você. Copie o exemplo e preencha:

```
cd ~/automic_auth_api
cp .env.example .env
$EDITOR .env
```

E carregue antes de qualquer comando do Kamal:

```
source .env
```

O `.env` está no `.gitignore`: confira antes de commitar. **Este é o arquivo que não pode vaziar.**

6. HTTPS na sua máquina

HTTPS é HTTP dentro de um túnel

O HTTP da Aula 1 é **texto puro**: quem estiver no caminho, o roteador do café ou o provedor, lê tudo, inclusive a senha. O **TLS** embrulha esse texto num túnel criptografado.

Mesmo protocolo, mesmos verbos, mesmos cabeçalhos, só que fechado. O "S" de HTTPS é isso, e nada além disso.

O handshake, em três passos

1. O servidor apresenta o **certificado** dele.
2. O cliente confere se confia em **quem assinou** aquele certificado.
3. Os dois combinam uma chave temporária e conversam com ela.

O par de chaves assimétricas só serve para o passo 3. Depois disso a conversa usa criptografia **simétrica**, que é muito mais rápida.

Certificado e autoridade

Um certificado diz: *"esta chave pública pertence a este domínio"*, e vem **assinado por uma Autoridade Certificadora (CA)**.

O seu navegador já nasce com uma lista de CAs em que confia. Confia na CA → confia em quem ela assinou. É uma cadeia.

- **Autoassinado** é você jurando que é você.
- **Let's Encrypt** é uma CA pública e gratuita, em que os navegadores confiam.

Para conseguir um certificado de CA pública é preciso **provar que o domínio é seu**, e você não tem domínio nenhum apontando para a sua máquina. Então hoje o certificado é autoassinado, e você vai ver, com os próprios olhos, o que isso significa.

Proxy reverso

Um proxy comum fica na frente do **cliente**. Um proxy **reverso** fica na frente do **servidor**: recebe tudo na 443, termina o TLS, e repassa para a aplicação em HTTP interno.

Assim o Rails não precisa saber nada de certificado:

```
seu navegador → kamal-proxy → Rails
```

É por isso que `config.assume_ssl = true`: ele avisa o Rails de que o "http" que chegou já veio de um "https" lá fora, e o Rails para de montar URLs erradas.

Gerar o certificado

Do seu notebook, na raiz do projeto:

```
openssl req -x509 -newkey rsa:2048 -sha256 -days 365 -nodes \
  -keyout tls/capacita-key.pem -out tls/capacita-cert.pem \
  -subj "/CN=seunome.test" \
  -addext "subjectAltName=DNS:seunome.test"
```

O `subjectAltName` não é opcional. Cliente moderno nenhum olha só o `CN`: sem SAN, o certificado é inválido mesmo estando certo.

E ponha o conteúdo nas variáveis, no seu `.env`:

```
KAMAL_PROXY_SSL_CERTIFICATE="$(cat tls/capacita-cert.pem)"
KAMAL_PROXY_SSL_PRIVATE_KEY="$(cat tls/capacita-key.pem)"
export KAMAL_PROXY_SSL_CERTIFICATE KAMAL_PROXY_SSL_PRIVATE_KEY
```

`.test` e o `/etc/hosts`

`seunome.test` não existe em DNS nenhum, e nunca vai existir: `.test` é um domínio **reservado para testes** justamente para isso ([RFC 6761](#)). Ninguém consegue registrar, então ele nunca vai colidir com um site de verdade.

Quem resolve esse nome é o seu próprio `/etc/hosts`, que o sistema consulta **antes** de perguntar ao DNS:

```
echo "SEU_IP seunome.test" | sudo tee -a /etc/hosts
```

```
getent hosts seunome.test # tem que devolver o IP da VM
```

Windows: repita no arquivo `C:\Windows\System32\drivers\etc\hosts`, com o Bloco de Notas aberto como administrador. É esse que o navegador do Windows consulta: o `/etc/hosts` do WSL2 vale só dentro do WSL2.

Autoassinado, na prática

Depois do deploy (seção 8), três comandos, na ordem:

Antes de rodar o primeiro, decida: o seu servidor está servindo HTTPS de verdade, com um certificado que você mesmo gerou. O `curl` vai funcionar ou vai reclamar? E se reclamar, vai ser por causa da criptografia ou por outra coisa?

```
curl https://seunome.test/api/v1/status
```

```
curl: (60) SSL certificate problem: self-signed certificate
```

A frase exata muda com a versão do `curl`: umas dizem `SSL certificate problem`, outras `SSL certificate OpenSSL verify result`, e as mais antigas escrevem `self signed` sem hífen. O que é sempre igual, e é o que importa, é o número **60** e a expressão *self-signed*.

Isso é o TLS funcionando, não falhando. O túnel subiu; o que falhou foi a *confiança*. O cliente não conhece quem assinou.

```
curl -k https://seunome.test/api/v1/status
```

O `-k` é "criptografa, mas não confere quem é". Funciona, e é exatamente o que você **não** faz em produção: sem verificar o certificado, qualquer um no meio do caminho pode se passar pelo servidor.

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

Aqui você não desligou nada: você **disse ao curl em quem confiar**. E é isso que uma CA é. Alguém em quem o seu sistema já decidiu confiar, de fábrica. Let's Encrypt não tem mágica nenhuma que o seu `openssl` não tenha; tem o navegador do mundo inteiro com a chave dela na lista.

Abra `https://seunome.test/api/v1/status` no navegador também. O aviso vermelho é o mesmo fato, dito para um humano.

7. GitHub Actions: o portão

`.github/workflows/ci.yml` roda em todo push e todo pull request:

- `scan_ruby`: Brakeman (segurança estática) e bundler-audit (CVE em gems)
- `lint`: RuboCop
- `test`: `bin/rails test` contra um Postgres descartável

Se qualquer um falhar, o código não entra na `main`.

Por que isso importa aqui, se o deploy é manual? Porque o CI não depende de onde o servidor mora. Ele é o portão: garante que o que você está prestes a colocar no ar compila, passa nos testes e não tem CVE conhecida. Um servidor sem CI publica bug automaticamente; um CI sem servidor ainda te avisa antes.

O deploy automático a cada push é o passo seguinte, e só faz sentido quando o servidor tem IP público: o runner do GitHub não alcança um IP dentro do seu notebook. Ele está pronto em `.github/workflows/deploy.yml`, e a seção **10** explica o que ele faz.

8. O primeiro deploy

Primeiro, valide sem tocar em nada:

```
cd ~/automic_auth_api
source .env
bundle exec kamal config
```

Sai um YAML grande, sem erro. Se reclamar de variável faltando, a mensagem diz exatamente qual.

Agora:

```
bundle exec kamal setup
```

`setup` instala o Docker se faltar, sobe os accessories e faz o primeiro deploy. **Só na primeira vez.** Depois é sempre `kamal deploy`.

Deu certo:

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

```
{"status":"ok","service":"automic-auth-api","environment":"production"}
```

O seu código está rodando dentro de um container, num Ubuntu que você provisionou, atrás de um proxy TLS, com um Postgres com volume. Em produção, é isto.

Daqui para frente, todo deploy é:

```
source .env && bundle exec kamal deploy
```

9. Operar

```
source .env

kamal app logs -f           # logs ao vivo
kamal app exec --interactive --reuse "bin/rails console"
kamal app exec "bin/rails db:migrate"
kamal app details           # o que está rodando
kamal accessory reboot db   # depois de mudar env do banco
kamal rollback              # volta para a versão anterior
```

Na máquina que serve, que hoje é a sua:

```
docker ps          # os containers do Kamal, rodando
docker logs -f automic-auth-api-db
free -h
df -h
```

Backup

Volume Docker não é backup. Um `docker volume rm` sem querer, ou a máquina que morre, leva o volume junto. Backup é uma cópia que vive **fora** dali.

```
docker exec automic-auth-api-db \
  pg_dump -U automic_auth_api automic_auth_api_production | gzip > backup-$(date +%F).sql.gz
```

E, o que quase ninguém faz: **restaure num banco de teste**. Backup que nunca foi restaurado não é backup, é esperança.

Parar e voltar

```
kamal app stop          # para a aplicação
kamal accessory stop db  # para o banco
kamal app boot           # traz de volta
```

Os containers ficam parados, não apagados. O volume do banco continua lá.

10. E num servidor de verdade

Tudo o que você fez hoje vale numa máquina alugada sem mudar de forma. O que **entra** são quatro coisas, e todas existem porque agora a máquina está exposta ao mundo:

	Hoje	Num servidor alugado
A máquina	já existe: é a sua	portal do provedor: região, tamanho, imagem, cota
Antes de tudo	nada	provisionar: usuário sem root, <code>apt upgrade</code> , Docker, swap
Firewall	nenhum, a máquina não está exposta	<code>ufw</code> dentro, mais o do provedor fora, com a 22 aberta só para o seu IP
SSH	você autorizou a si mesmo	endurecer o <code>sshd</code> : sem senha, sem root
Nome	<code>/etc/hosts</code>	DNS de verdade, num domínio seu
Certificado	autoassinado	emitido por uma CA, que exige provar que o domínio é seu
Deploy	<code>kamal deploy</code> no seu terminal	GitHub Actions, por OIDC, sem senha guardada

E no `.env`, isso é:

```
export SERVER_IP=57.156.65.151    # em vez de 127.0.0.1
export KAMAL_SSH_USER=ubuntu      # em vez do seu login
```

Duas linhas. O resto do arquivo, o `deploy.yml`, o `Dockerfile` e os comandos são os mesmos.

O deploy automático

O `.github/workflows/deploy.yml` deste repositório é o pipeline pronto. A sequência dele:

1. autentica na nuvem (OIDC)
2. descobre o próprio IP público
3. cria uma regra no firewall liberando a 22 só para esse IP/32
4. faz o deploy
5. APAGA a regra, mesmo se o deploy falhar

O passo 5 tem `if: always()`. **Deixar a 22 aberta é o erro que transforma um deploy ruim num incidente de segurança.**

E o **OIDC** é o motivo de não haver senha nenhuma nisso: em vez de guardar um segredo de longa duração no GitHub, o GitHub emite a cada execução um token curto que diz "sou o workflow do repositório X, na branch `main`", e a nuvem devolve um acesso temporário.

Isto é a demonstração de hoje, não o exercício. O passo a passo completo, com uma VM na Azure, o firewall, o DNS no Cloudflare, o certificado e a credencial federada do OIDC, está em [apendice-azure.md](#), para você fazer em casa com a [Azure for Students](#): US\$100 de crédito, sem cartão.

As práticas da aula

Sete práticas, cada uma logo depois do bloco que a explica. Nesta aula um passo mal feito só aparece três passos depois, então **não siga com uma conferência vermelha**.

#	O que	Depois de
1	A sua máquina vira o servidor	seção 2
2	Os arquivos de deploy e o token do registry	seção 3
3	O <code>.env</code>	seção 5
4	O certificado e o <code>/etc/hosts</code>	seção 6
5	O primeiro deploy	seção 8
6	O certificado, com os olhos	seção 8
7	O sistema inteiro, e o backup	seção 9

Prática 1: a sua máquina vira o servidor

Se você usa Windows, tudo isto é **dentro do WSL2**. É ele o seu Linux.

```
# a) o servidor de SSH
sudo apt update && sudo apt install -y openssh-server      # Ubuntu, Debian, WSL2
sudo service ssh start
# macOS: Ajustes do Sistema → Geral → Compartilhamento → Sessão remota

# b) o par de chaves da capacitação
ssh-keygen -t ed25519 -f ~/.ssh/capacita -C "capacita-servidor" -N ""
chmod 600 ~/.ssh/capacita

# c) autorizar a si mesmo
mkdir -p ~/.ssh && chmod 700 ~/.ssh
cat ~/.ssh/capacita.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

O que você deve ver:

```
ssh -i ~/.ssh/capacita -o IdentitiesOnly=yes "$USER"@127.0.0.1 'echo SSH_OK && docker -v'
```

Tem que sair `SSH_OK` e a versão do Docker. Se saiu, o Kamal tem tudo de que precisa.

Se der errado

Erro	Causa	Saída
<code>Connection refused</code> na 22	o <code>sshd</code> não está rodando	<code>sudo service ssh start</code> ; no macOS, ligue a Sessão remota
<code>Connection refused</code> depois de reiniciar o Windows	o WSL2 não sobe o <code>sshd</code> sozinho	<code>sudo service ssh start</code> de novo. É o tropeço mais comum do dia
<code>Permission denied (publickey)</code>	permissão frouxa na chave	<code>chmod 600 ~/.ssh/capacita</code> e <code>chmod 700 ~/.ssh</code>
<code>Permission denied</code> mesmo com <code>chmod</code>	a pública não entrou no <code>authorized_keys</code>	<code>cat ~/.ssh/authorized_keys</code> e confira que a linha está lá
<code>Too many authentication failures</code>	o SSH ofereceu todas as chaves do agente	acrescente <code>-o IdentitiesOnly=yes</code>
<code>Host key verification failed</code>	a chave do host mudou	<code>ssh-keygen -R 127.0.0.1</code> e conecte de novo
<code>docker: command not found</code> pela SSH	o <code>PATH</code> da sessão não interativa é menor	confirme com <code>ssh ... 'which -a docker'</code> ; se não achar, reinstale o Docker pelo pacote do sistema
<code>sudo</code> pede senha e trava o comando	esperado na primeira vez	digite a senha; é só a instalação

Prática 2: os arquivos de deploy e o token do registry

```
cd ~/capacitacao-gabarito && git checkout aula-04
rsync -aR config/deploy.yml config/environments/production.rb \
    .kamal scripts .github .env.example Dockerfile .dockerignore Gemfile Gemfile.lock \
    ~/automic_auth_api/

cd ~/automic_auth_api && bundle install
```

O `-R` não é detalhe: sem ele o `rsync` joga `deploy.yml` e `production.rb` na raiz do projeto, fora de `config/`, e o Kamal não acha nada.

O seu projeto já tinha um `config/deploy.yml`, um `Dockerfile` e um `.kamal/`, gerados pelo `rails new`. Você está **substituindo** o `deploy.yml` genérico pelo nosso.

Crie o **PAT (classic)** no GitHub: **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token**, com `write:packages` e `read:packages`. Ele é mostrado **uma vez**.

O que você deve ver:

```
ls config/deploy.yml .kamal/secrets .env.example
uname -m           # x86_64 é amd64; aarch64 ou arm64 é arm64. Anote para o .env
```

Se der errado

Erro	Causa	Saída
<code>deploy.yml</code> foi parar na raiz	faltou o <code>-R</code>	apague e refaça o <code>rsync</code> com <code>-R</code>
<code>.kamal/</code> não veio	pasta oculta	o comando lista <code>.kamal</code> explicitamente; copie-o inteiro
<code>Could not find gem 'kamal'</code>	faltou <code>bundle install</code>	rode
<code>git checkout aula-04</code> reclama de alterações locais	you editou o gabarito	<code>git -C ~/capacitacao-gabarito checkout .</code> ; o gabarito é só leitura
o PAT sumiu da tela	é mostrado uma vez só	gere outro; não há como recuperar
criou um token <i>fine-grained</i>	o ghcr.io não aceita	tem que ser classic

Prática 3: o `.env`

Errar aqui é o que causa quase toda falha da Prática 5.

```
cd ~/automic_auth_api
cp .env.example .env
$EDITOR .env
source .env
```

O `.env.example` já vem com `SERVER_IP=127.0.0.1` e `KAMAL_SSH_USER="$USER"` preenchidos, porque o servidor é a sua máquina. O `SERVER_ARCH` é o `uname -m` da prática anterior, traduzido: `x86_64` vira `amd64`, e `aarch64` ou `arm64` viram `arm64`.

O que falta você preencher: `GHCR_USER`, `SERVER_ARCH`, `APP_HOST`, `KAMAL_REGISTRY_PASSWORD` (o PAT), `RAILS_MASTER_KEY` (o conteúdo de `config/master.key`), `AUTOMIC_AUTH_API_DATABASE_PASSWORD`, `JWT_SECRET` (saída de `bin/rails secret`), `CORS_ORIGINS`, `MAILER_FROM` e os três `SMTP_*`.

O que você deve ver:

```
git status # o .env NÃO pode aparecer
bundle exec kamal config > /dev/null && echo "config ok"
```

Se o `.env` aparecer no `git status`, **pare tudo** e conserte o `.gitignore` antes de qualquer commit. Segredo commitado não sai do histórico apagando o arquivo.

Se der errado

Erro	Causa	Saída
<code>key not found: "GHCR_USER"</code>	esqueceu o <code>source .env</code>	<code>source .env</code> , e ele vale só naquele shell
<code>cat: config/master.key: No such file</code>	o Rails ainda não gerou	<code>bin/rails credentials:edit</code> cria; feche o editor para salvar
<code>kamal config</code> reclama de outra variável	ela está vazia no <code>.env</code>	a mensagem diz o nome exato
o <code>.env</code> aparece no <code>git status</code>	falta a linha <code>.env</code> no <code>.gitignore</code>	acrescente antes de commitar
<code>\$EDITOR: command not found</code>	variável não definida	<code>nano .env</code> ou <code>code .env</code>

Prática 4: o certificado e o `/etc/hosts`

```
mkdir -p tls
openssl req -x509 -newkey rsa:2048 -sha256 -days 365 -nodes \
  -keyout tls/capacita-key.pem -out tls/capacita-cert.pem \
  -subj "/CN=seunome.test" -addext "subjectAltName=DNS:seunome.test"

echo "127.0.0.1 seunome.test" | sudo tee -a /etc/hosts
getent hosts seunome.test
```

O que você deve ver: o `getent` devolve `127.0.0.1`, e o certificado tem o SAN certo:

```
openssl x509 -in tls/capacita-cert.pem -noout -text | grep -A1 "Subject Alternative Name"
```

Windows: para abrir no navegador do Windows, repita a linha em `C:\Windows\System32\drivers\etc\hosts`, com o Bloco de Notas aberto como administrador. O WSL2 espelha o `localhost` para o Windows, então `127.0.0.1` funciona dos dois lados.

Se der errado

Erro	Causa	Saída
<code>unknown option -addext</code>	OpenSSL 1.0, antigo	atualize, ou use um arquivo de config com <code>[v3_req]</code>
<code>getent</code> não devolve nada	a linha do <code>/etc/hosts</code> saiu torta	tem que ser <code>IP<espaço>nome</code> , sem vírgula
o navegador do Windows não acha o nome	falta a linha no hosts do Windows	veja a nota acima
sem "Subject Alternative Name" na saída	faltou o <code>-addext</code>	gere de novo: sem SAN, cliente moderno nenhum aceita
a chave foi para o Git	<code>.pem</code> de chave é segredo	acrescente <code>tls/*-key.pem</code> ao <code>.gitignore</code>

Prática 5: o primeiro deploy

```
source .env
bundle exec kamal config      # valida sem tocar em nada
bundle exec kamal setup      # só na primeira vez
```

O que você deve ver:

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

```
{"status":"ok","service":"automic-auth-api","environment":"production"}
```

Pare um segundo aqui. O seu código está rodando dentro de um container, com o Rails em modo produção, atrás de um proxy que termina TLS, falando com um Postgres que tem volume. Você não está mais rodando `bin/rails server`.

Se der errado

Esta é a maior tabela da capacitação, de propósito: o primeiro deploy cobra de uma vez tudo o que você configurou hoje. Se falhar, **quase nunca é o Kamal**. É uma variável do `.env`, o tipo do token, ou a arquitetura. Leia a mensagem, ache a linha aqui, corrija, e rode de novo: `kamal setup` duas vezes não estraga nada.

Erro	Causa	Saída
<code>denied</code> / <code>unauthorized</code> ao empurrar a imagem	PAT sem <code>write:packages</code> , ou <i>fine-grained</i>	gere um classic com os dois escopos
erro de cota ou de armazenamento no <code>push</code>	a conta grátis limita pacote privado	deixe o pacote público (seção 3), e apague versões antigas em Packages
<code>GHCR_USER</code> recusado pelo registry	maiúscula no nome	o ghcr.io só aceita minúsculas
<code>exec format error</code> no container	arquitetura errada	<code>uname -m</code> e ajuste <code>SERVER_ARCH</code>
<code>Docker is not installed</code>	you rodou <code>deploy</code> em vez de <code>setup</code>	o primeiro é sempre <code>setup</code>
<code>Connection refused</code> na 22	o <code>sshd</code> parou	<code>sudo service ssh start</code>
<code>Host key verification failed</code>	primeira conexão do Kamal	<code>ssh -i ~/.ssh/capacita "\$USER"@127.0.0.1</code> uma vez e aceite
<code>address already in use</code> na 80 ou 443	outra coisa ocupa a porta	<code>sudo ss -tlnp grep -E ':(80 443)'</code> e libere
a aplicação sobe mas não acha o banco	<code>kamal deploy</code> não sobe accessories	<code>kamal accessory boot db</code>
<code>404</code> do proxy	<code>APP_HOST</code> diferente do nome que você chamou	os dois têm que bater exatamente
health check falhando em loop	a aplicação estoura ao subir	<code>kamal app logs -f</code> : quase sempre <code>RAILS_MASTER_KEY</code> errada ou migration pendente
o build demora e a máquina engasga	build de imagem consome bastante	feche o resto; a primeira vez é sempre a mais lenta

Prática 6: o certificado, com os olhos

Três comandos. Cada um responde uma parte da pergunta "o que o HTTPS garante, e o que não".

Antes de rodar o primeiro, decida: o seu servidor está servindo HTTPS de verdade, com um certificado que você mesmo gerou. O `curl` vai funcionar ou vai reclamar? E se reclamar, vai ser por causa da criptografia ou por outra coisa?

```
curl https://seunome.test/api/v1/status
```

Tem que **falhar**, com o erro **60** e a expressão *self-signed*. **Isso é o TLS funcionando, não falhando**: o túnel subiu, e o que faltou foi a confiança.

```
curl -k https://seunome.test/api/v1/status
```

Funciona. O `-k` é "criptografa, mas não confere quem é".

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

Funciona **conferindo**. Você não desligou nada: você disse ao `curl` em quem confiar.

Abra também no navegador e leia o aviso.

O que você deve ver: escreva numa linha, para você mesmo, a diferença entre o segundo e o terceiro comando. Se conseguir, você entendeu o que uma CA é.

Se der errado

Erro	Causa	Saída
o primeiro comando funcionou	you already trusted this certificate on this machine	é raro; confira com outro nome em <code>.test</code>
<code>unable to get local issuer certificate</code>	mensagem diferente, mesmo fato	é a mesma lição: falta confiança
o navegador não deixa passar de jeito nenhum	HSTS de um teste anterior	use aba anônima, ou outro nome em <code>.test</code>

Prática 7: o sistema inteiro, e o backup

- [] Refaça o fluxo da Aula 3 **contra a sua API em produção** (`https://seunome.test`), e desta vez o e-mail chega de verdade na sua caixa de entrada, não no navegador.
- [] Mude a mensagem da rota de status, commite, dê push (o CI roda) e:

```
source .env && bundle exec kamal deploy
kamal app logs -f
```

- [] Faça um backup, e confira que não está vazio:

```
docker exec automic-auth-api-db \
  pg_dump -U automic_auth_api automic_auth_api_production | gzip > backup-$(date +%F).sql.gz
ls -lh backup-*.sql.gz
```

- [] Quando quiser liberar a máquina: `kamal app stop` para o app, e `kamal accessory stop db` para o banco. `kamal app boot` traz de volta.

O que você deve ver: o backup tem mais que alguns bytes, e o `kamal app logs` mostrou o container novo subindo ao lado do antigo antes de trocar.

Se der errado

Erro	Causa	Saída
o e-mail não chega	SMTP não configurado, ou porta bloqueada	confira os <code>SMTP_*</code> do <code>.env</code> ; algumas redes bloqueiam a 587
o backup saiu com 20 bytes	o <code>pg_dump</code> falhou e o <code>gzip</code> comprimiu o vazio	tire o <code>\ gzip</code> e leia o erro
<code>docker exec</code> não acha o container	nome diferente	<code>docker ps</code> e use o nome que aparece
o segundo deploy não mudou nada	you não commitou antes	o Kamal usa o commit atual como versão da imagem
<code>kamal deploy</code> diz que há um lock	um deploy anterior morreu no meio	<code>kamal lock release</code>

Bônus, se sobrou tempo

- Derrube o app de propósito (`kamal app stop`) e veja o que o `curl` responde. Depois `kamal app boot`.
- Faça um deploy que **falha** no health check (quebre a rota `/up`) e confirme que o Kamal **não** troca o tráfego: a versão antiga continua servindo.
- Rode `kamal rollback` e veja voltar para a imagem anterior.
- Abra o `apendice-azure.md` e provisione uma máquina de verdade. O `deploy.yml` é o mesmo: muda o `SERVER_IP` e o `KAMAL_SSH_USER`.

Recapitulando

- Publicar não é "rodar num lugar diferente": é empacotar, versionar, entregar e conseguir voltar atrás. Que a máquina seja a sua ou a de um datacenter muda duas linhas do `.env`.
- Escolha a ferramenta do tamanho do problema. Kubernetes não era o tamanho.
- Imagem é receita, container é bolo. Multi-stage e usuário não-root.
- `clear` é configuração, `secret` é segredo, e a diferença aparece no `docker inspect`.
- Volume é o que faz o dado sobreviver ao deploy. E ainda assim não é backup.
- TLS criptografa; **CA é confiança**. São coisas separadas, e o `curl -k` mostra a costura.
- CI é o portão. Ele te avisa antes, com ou sem deploy automático.

- Numa máquina exposta, provisionar direito, o firewall e o segredo de curta duração deixam de ser detalhe. É o assunto do apêndice.

O que o `seem-backend` tem a mais

O que ficou de fora daqui, e por quê:

Recurso	Para quê
Datadog (logs)	logs centralizados e Error Tracking, com o app logando JSON estruturado
rack-attack	bloqueia scanner e rajada de erro por IP: a internet bate na sua porta o dia todo
Active Storage	foto de perfil, em volume Docker persistente
Solid Queue dedicado	processamento em background
Audit log	histórico de toda mutação feita por admin
Export .xlsx	relatório de presença
OpenAPI/Swagger	contrato da API documentado

Nenhum deles é difícil depois do que você viu aqui. Todos estão no `seem-backend`. Agora dá para ler.